

上位机通信

上位机通信是工业自动化、物联网、嵌入式系统等领域中非常关键的技术环节，主要用于主控计算机（上位机）与终端设备（下位机、如PLC、传感器、执行器、智能仪表等）之间的数据交互。以下从概念、通信过程、常见通信方式、注意事项等方面进行说明：

上位机通信的概念

1.上位机 (Host Computer)

- 通常是高性能计算机或工业PC，负责监控、数据采集、逻辑控制和人机交互(HMI)。
- 功能示例:接收传感器数据、下发控制指令、生成生产报表、报警处理等。

2.下位机 (Slave Device)

- 指直接连接物理设备的终端控制器，如PLC、单片机、嵌入式系统等。
- 功能示例:采集温度信号、控制电机启停、执行逻辑运算等。

3.通信的本质

- 上位机与下位机通过特定的协议和硬件接口，实现双向数据交换。
- 典型场景：
 - 数据上传:下位机将设备状态(如温度、压力)发送给上位机。
 - 指令下发:上位机发送控制命令(如启动/停止设备)到下位机。

二、上位机通信的基本过程

1.建立通信连接

- 硬件接口选择:根据需求选择物理连接方式:
 - 串口通信:RS-232、RS-485(低成本、短距离)。
 - 以太网:TCP/IP、Ethernet/IP(高速、远距离)。
 - 工业总线:CAN、EtherCat、Profinet(高实时性)。
 - 无线通信:Wi-Fi、蓝牙、LoRa(灵活部署)。
- 协议配置
 - 协议类型:Modbus、OPC UA、CANopen、EtherCAT、PLC种协议等。
 - 参数匹配:波特率(如9600bps)、数据位(8位)、校验方式(奇偶校验)等需与下位机一致。

2.数据交换

- 数据格式:
 - 字节流:原始二进制数据(需按协议解析)。

```
byte[] data = new byte[]{0xe1,ex02,exe3,exe4};
//ModbusRTU协议
byte[] data = new byte[]{0x01,0x03,0x00,0x00, 0x00, 0x0A,0xC5,0xcD};
```

- 结构化数据:JSON、XML(可读性高，常用于物联网)。
- 通信模式：
 - 轮询(Polling):上位机主动请求数据(简单但实时性低)。
 - 中断(Interrupt):下位机主动上报事件(实时性高)。
 - 发布-订阅(Pub/Sub):适用于多设备协同场景(如MQTT)

3.异常处理

- 超时重试:若未收到响应，自动重发请求(需设置最大重试次数)。
- 错误校验：
 - 硬件校验:奇偶校验、CRC校验。
 - 软件校验:校验和(Checksum)、哈希值。
- 日志记录:记录通信失败的时间、错误码、数据内容，便于故障排查。

4.关闭连接

- 正常关闭:发送终止指令，释放资源。
- 异常恢复:断线后自动重连(需心跳包机制)

5.附表:常见的通信方式与协议

协议方式	特点	典型应用场景
Modbus	-简单易用，支持串口(RTU)和以太网(TCP)-主从架构，功能码明确	PLC与上位机通信、工业仪表数据采集
OPC UA	跨平台、高安全性-支持复杂数据结构和历史数据访问	工业4.0、SCADA系统
CAN/CANopen	-高抗干扰性，适合恶劣环境-多主节点通信，优先级仲裁机制	汽车电子、机器人控制
EtherCAT	高实时性，微秒级响应-分布式时钟同步	运动控制、精密加工设备
MQTT	轻量级，基于发布-订阅模式-适合低带宽、不稳定网络环境	物联网(IoT)、远程监控

三、注意事项

1.实时性与性能

- 实时性要求：
 - 运动控制、机器人等场景需保证通信周期(如EtherCAT的1ms周期)。
 - 避免因数据处理延迟导致控制失效(如使用高优先级线程)。

- 带宽规划:
 - 估算数据量(如每秒100个传感器的4字节数据, 需至少400B/s带宽)。

2.数据安全

- 加密传输: 敏感数据(如设备密码)使用TLS/SSL加密。
- 权限控制: 限制非法设备接入(如MAC地址白名单、OPCUA证书认证)

3.错误处理与鲁棒性

- 断线重连: 实现心跳包机制(如每5秒发送一次心跳信号)。
- 数据缓存: 网络中断时, 本地缓存未发送的数据, 恢复后补发。

4.兼容性与标准化

- 协议兼容: 确保上位机支持下位机的协议版本(如Modbus TCP与RTU的差异)。
- 接口统一: 使用OPCUA等标准化协议, 避免私有协议导致的系统封闭。

5.开发与调试

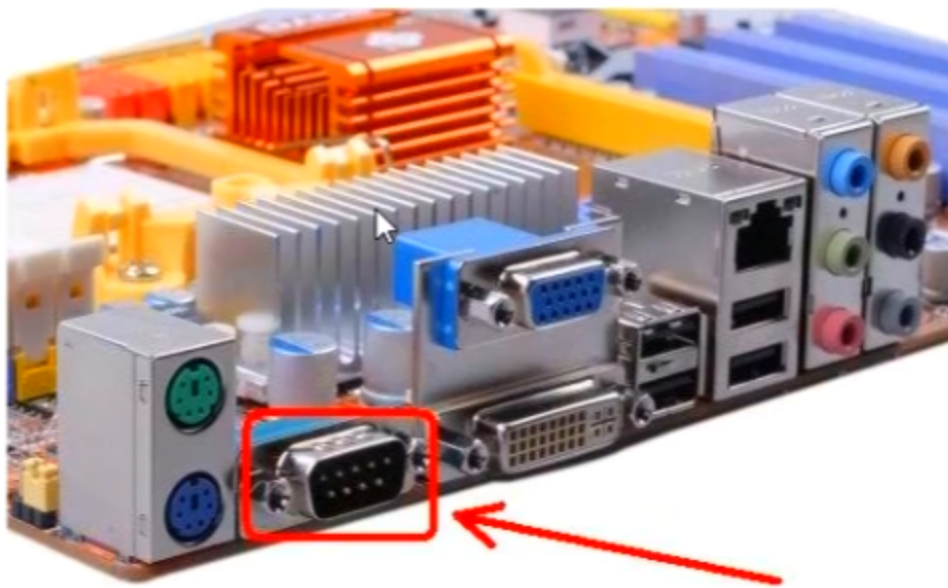
- 调试工具:
 - 串口调试助手: 验证串口通信过程, 以及相关数据帧。
 - 网络调试助手: 验证以太网通信过程, 以及相关数据帧
- 模拟测试: 用虚拟下位机(如Modbus Slave模拟软件)提前验证逻辑。

串口通信与协议

一、串口学习环境

1.上位机接口

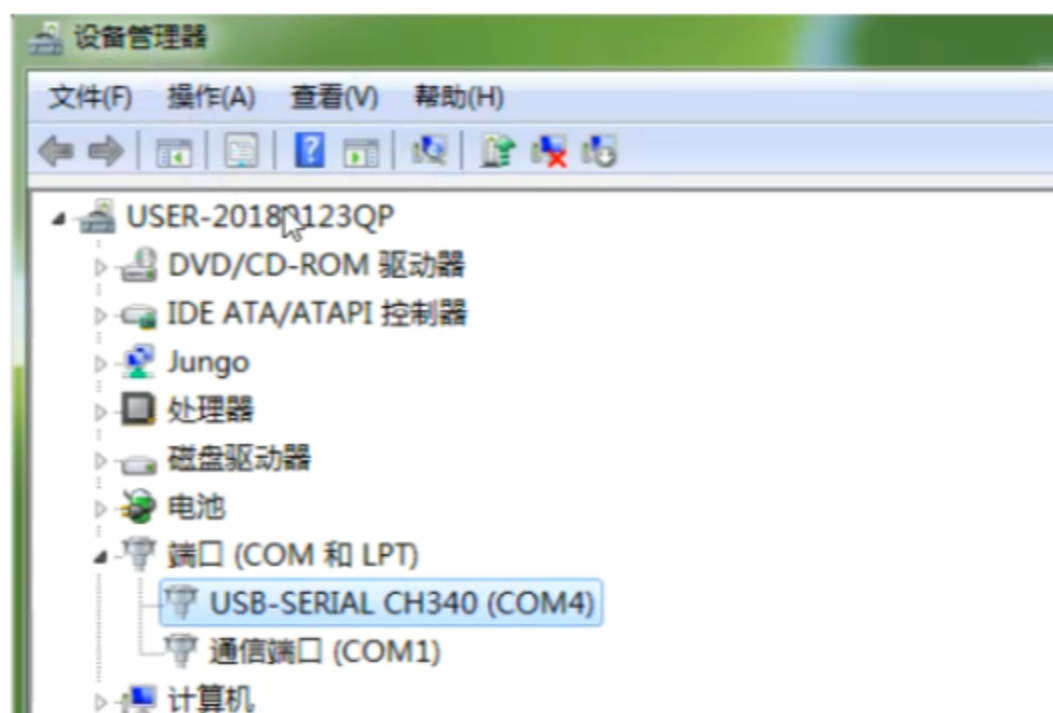
- 主板接口硬件:



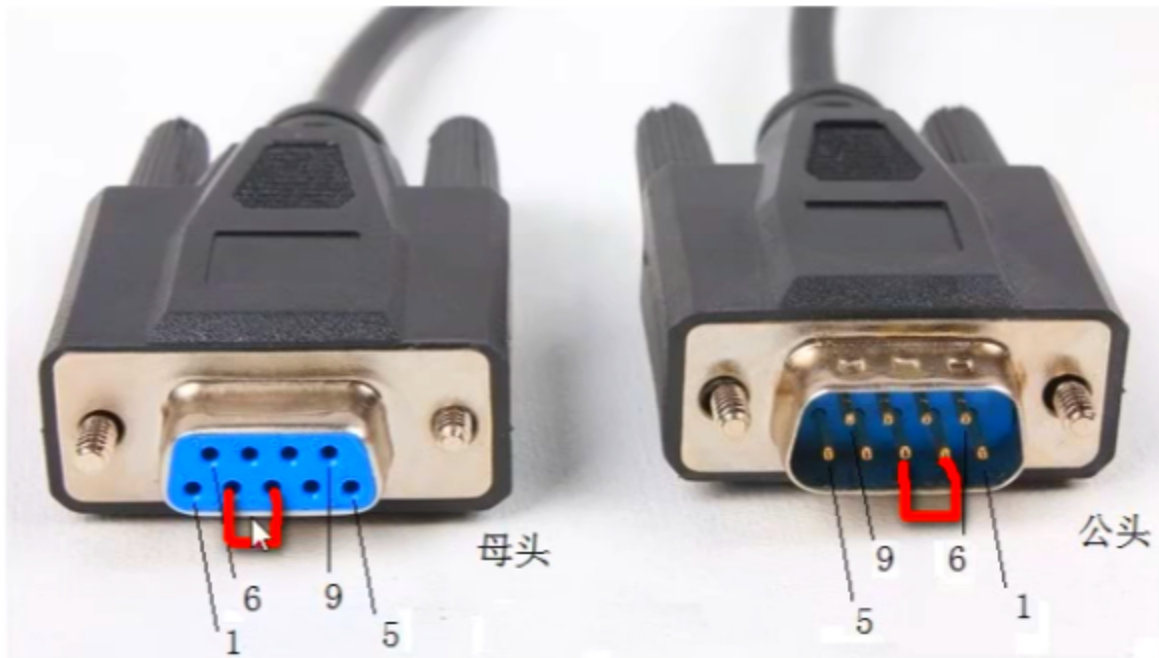
- USB转串口



- 检查串口



- 接口短接（自通信测试）



2.虚拟串口

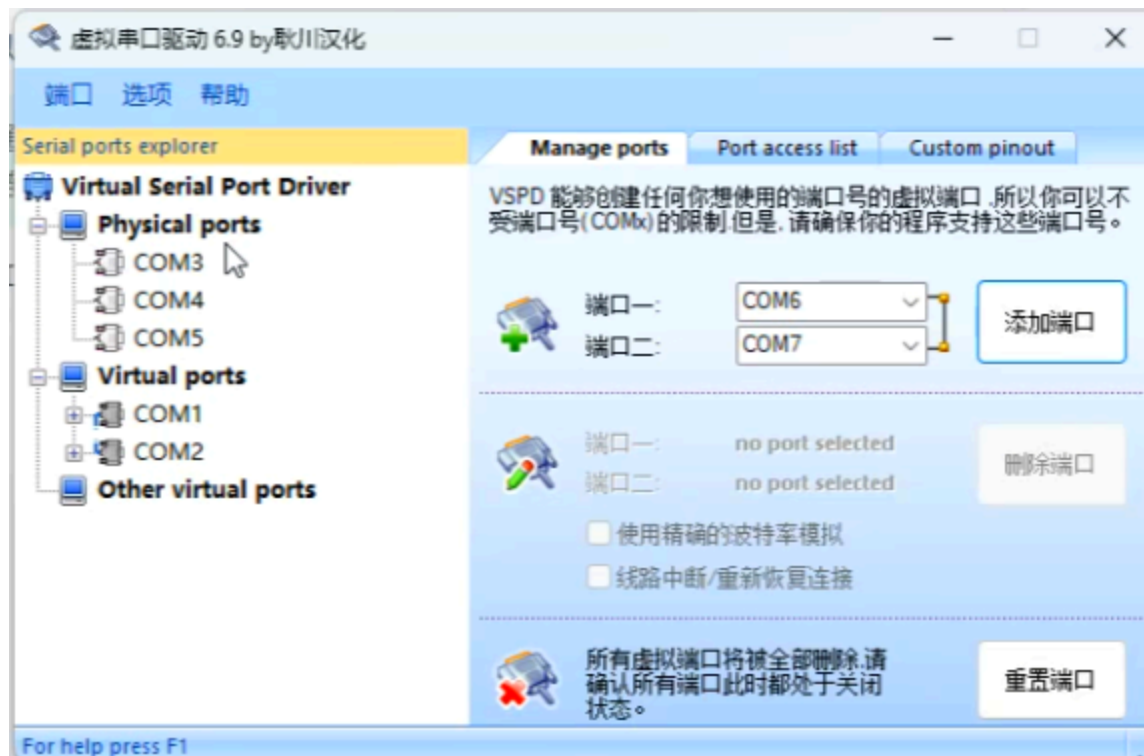
虚拟串口（VSPD）软件安装

网址：

通过网盘分享的文件:VSPD6.9.rar

链接:[VSPD 6.9.rar_免费高速下载|百度网盘-分享无限制](#)

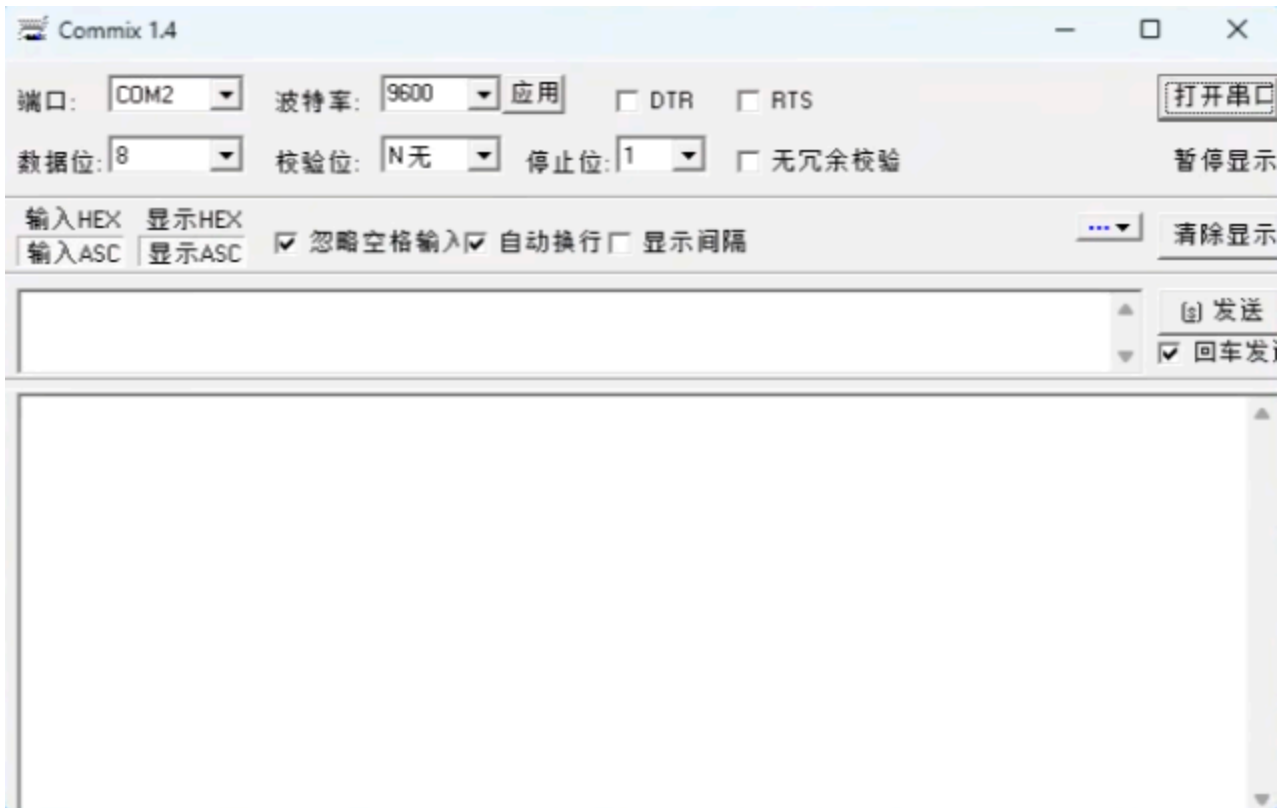
VSPD软件使用，添加成对虚拟串口



3. 串口调试助手

通过网盘分享的文件:Commix.rar

链接: <https://pan.baidu.com/s/1fnFsl-rAn5qQ1JousNTVFg?pwd=vbqx> 提取码:ybgx



二、串口对象SerialPort

1. SerialPort

```
//对象实例化
SerialPort serialPort = new SerialPort("COM1", 9600, Parity.Even, 8,
StopBits.One)
{
    ReadTimeout = 1000, //读取超时
    WriteTimeout = 1000 //写入超时

};
serialPort.Open(); //打开串口
```

2. 核心属性与方法

属性/方法	说明	示例
PortName	串口号 (如COM3)	<code>serialPort.PortName = "COM3"</code>
BaudRate	波特率 (如9600)	<code>serialPort.BaudRate = 9600</code>
DataBits	数据位 (5~9, 常用8)	<code>serialPort.DataBits = 8</code>
Parity	校验位 (None、Even、Odd)	<code>serialPort.Parity = Parity.None</code>
StopBits	停止位 (One、Two)	<code>serialPort.StopBits = StopBits.One</code>
ReadBufferSize	输入缓冲区的大小	<code>serialPort.ReadBufferSize = 4800</code>
ReadTimeout	读取操作未完成时发生超时之前的毫秒数	<code>serialPort.ReadTimeout= 2000</code>
WriteBufferSize	串行端口输出缓冲区的大小	<code>serialPort.WriteBufferSize= 4800</code>
WriteTimeout	写入操作未完成时发生超时之前的毫秒数	<code>serialPort.WriteTimeout= 2000</code>
Open()	打开串口	<code>serialPort.Open()</code>
Close()	关闭串口	<code>serialPort.Close()</code>
Write()	发送字节数组	<code>serialPort.Write(data, 0, data.Length)</code>
ReadExisting()	读取接收缓冲区所有字符	<code>string received = serialPort.ReadExisting()</code>

3.事件驱动通信

- 关键事件:DataReceived(数据到达时触发)
- 实现异步接收:

```
serialPort.DataReceived += (sender, e) =>
{
    if (e.EventType == SerialData.Chars)
    {
        string data = serialPort.ReadExisting();
        Console.WriteLine($"收到数据:{data}");
    }
};
```

4.注意事项

- 线程安全:: DataReceived 事件在非U线程触发, 需通过 Invoke 更新界面。

- 资源释放：使用 using 或 try/finally 确保串口关闭。
- 缓冲区管理：及时读取数据，避免缓冲区溢出。

三、Modbus协议

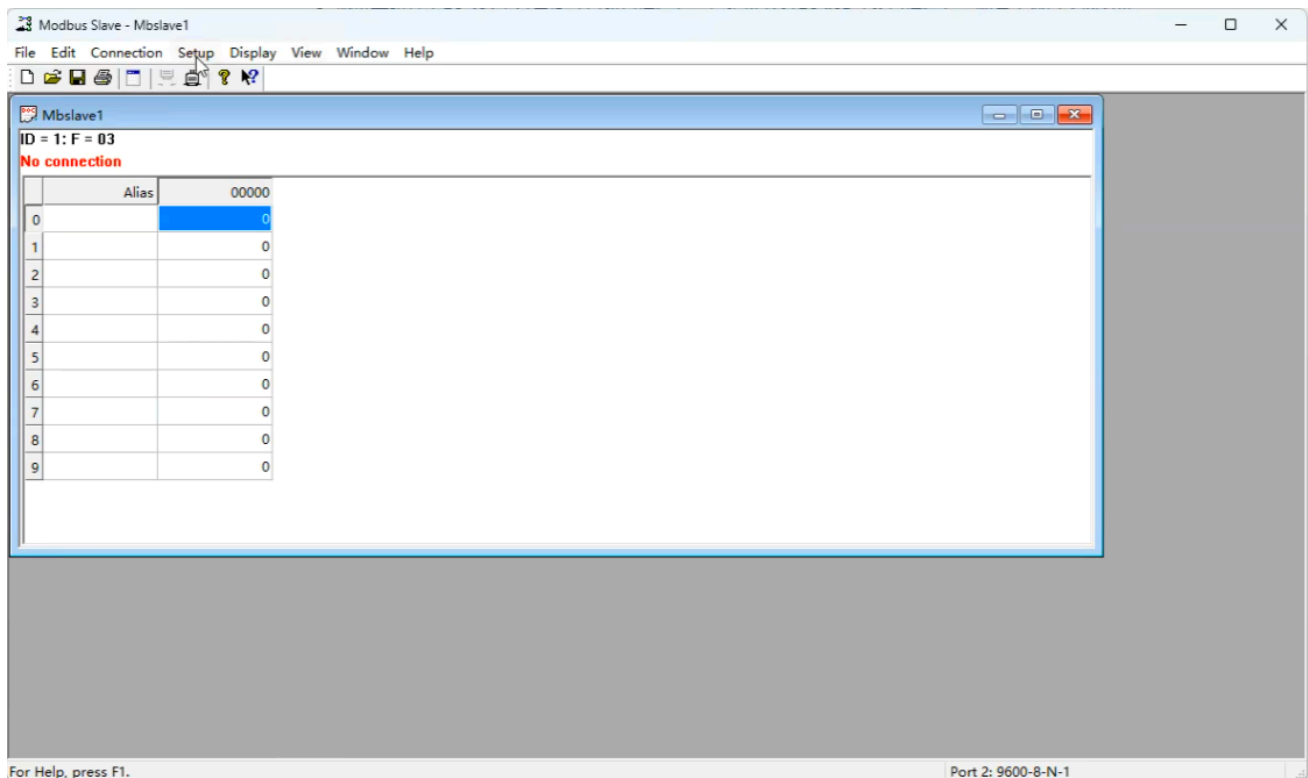
- Modbus是一种由Modicon公司(现为施耐德电气Schneider Electric)于1979年推出的**串行通信协议**，最初设计用于可编程逻辑控制器(PLC)间的通信。
- 作为工业自动化领域的事实标准(Defacto)，Modbus已成为电子设备间最常用的连接协议之一。2006年，其被国际电工委员会(IEC)收录为IEC61158国际标准，进一步巩固了行业地位。
- 协议最初基于RS-232串行总线(点对点通信)，后扩展支持RS-485(多点通信)。随着工业以太网发展，Modbus衍生出面向TCP/IP网络的Modbus TCP变体，同时保留了Modbus RTU(二进制高效传输)和Modbus ASCII(可读文本格式)两种串行通信模式。

1.调试环境

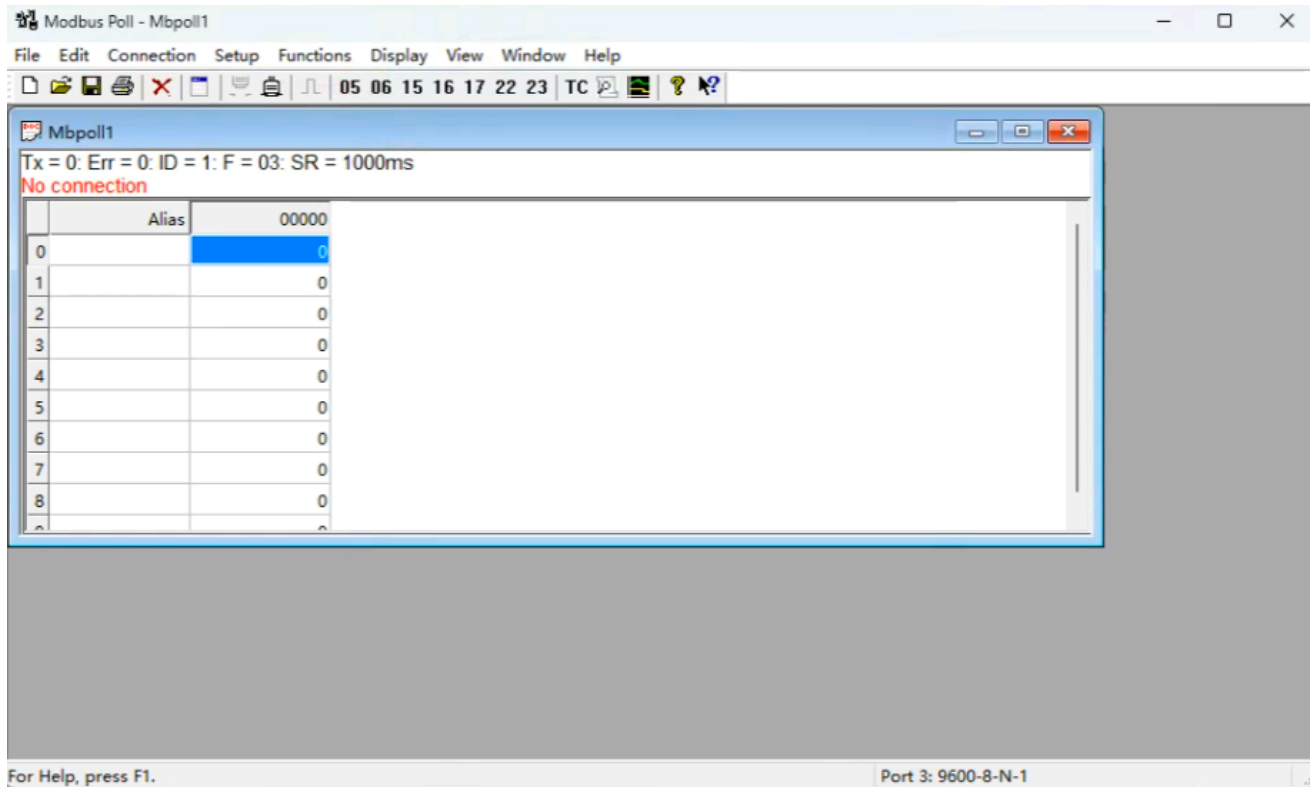
通过网盘分享的文件:Modbus三件套.rar

链接: <https://pan.baidu.com/s/18H0QvZHdmw4-geASvvv07A?pwd=49aq> 提取码: 49aq 复制这段内容后打开百度网盘手机App，操作更方便哦

- ModbusSlave软件安装（Modbus从站模拟）



- ModbusPoll软件安装（Modbus主站模拟）



注意:

Modbus Slave:Modbus从站模拟

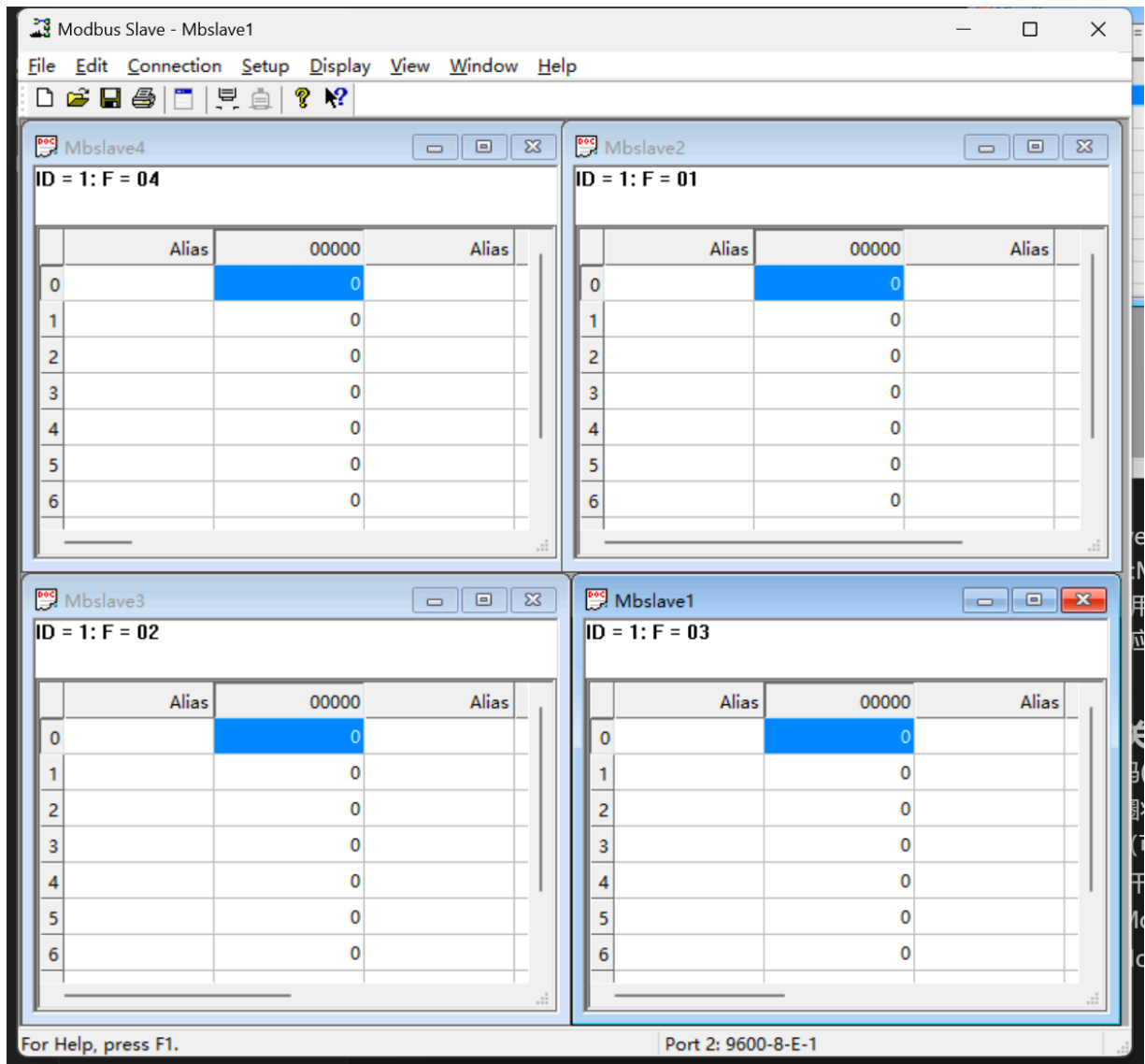
Modbus Poll:Modbus主站模拟

第一次打开使用功能的时候(比如Setup菜单、比如Connection菜单), 会触发序列号窗口, 此时将对应压缩包里的SN字符串复制进去即可!!

2.Modbus关键概念

- 从站设备编码(从站地址、单元ID)，一主多从
- 存储区：线圈状态（bool，0/1，true/false）、输入线圈(只读)、输入寄存器(只读)、保持型寄存器（可读可写）

- 协议公开的，设备可以支持



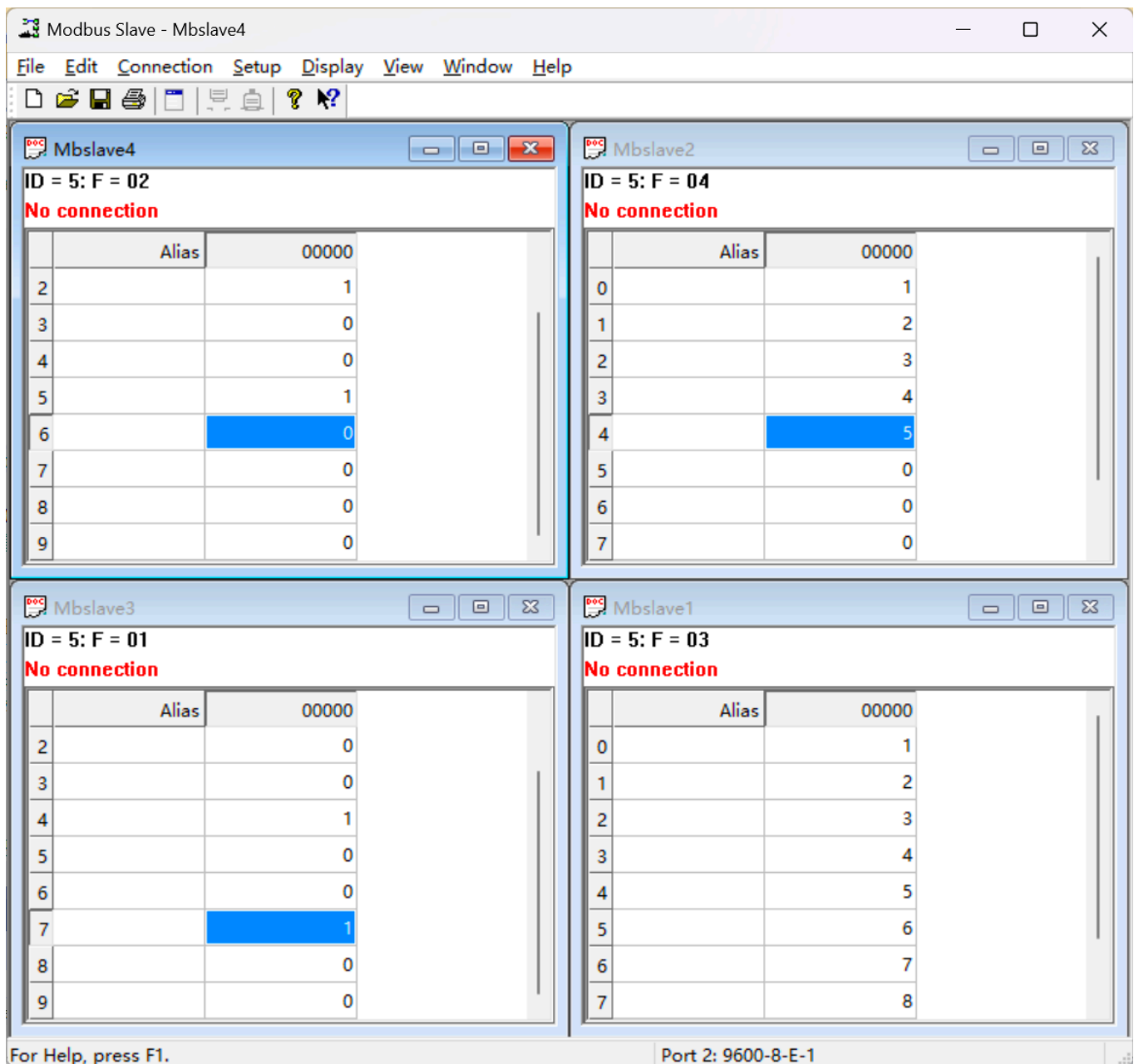
- 1-65535
- 功能码：
 - 线圈状态（01-读写，05-单写，15-多写）
 - 输入线圈（02-读取）
 - 输入寄存器（04-读取）
 - 保持型寄存器（03-读取，06-单写，16-多写）

- 地址案例，数据点表

Modbus 地址	PLC 地址	变量名称	备注说明
40001	DB1.D8W0	正向点动	1=True,0=False
40002	DB1.D8W2	反向点动	1=True,0=False
40003	DB1.D8W4	相对运动	1=True,0=False
40004	DB1.D8W6	绝对运动	1=True,0=False
40005	DB1.D8W8	回原点	1=True,0=False
40006	DB1.D8W10	系统复位	1=True,0=False
40007	DB1.D8W12	系统暂停	1=True,0=False
40008-40009	DB1.D8D14	点动速度	浮点型
40010-40011	DB1.D8D18	相对运动速度	浮点型
40012-40013	DB1.D8D22	相对运动距离	浮点型
40014-40015	DB1.D8D26	绝对运动速度	浮点型
40016-40017	DB1.D8D30	绝对运动位置	浮点型
40018-40019	DB1.D8D34	实时位置	浮点型

- 协议分类：ModbusRTU、ModbusASCII、ModbusTCP
- 通信库：NModbus4(Nuget安装)

3.ModbusRTU



```
//创建ModbusRTU通信协议对象
```

```
IModbusSerialMaster master = ModbusSerialMaster.CreateRtu(serialPort);
```

```
master.Transport.ReadTimeout = 2000; //读超时
```

```
master.Transport.Retries = 3; //重试次数
```

```
//读取从站5的保持型寄存器0~9
```

```
ushort[] registers = master.ReadHoldingRegisters(
    slaveAddress: 5, //从站地址 (1-247)
    startAddress: 0, //起始寄存器地址
    numberOfPoints: 10 //读取寄存器数量
);
```

```
//异步读取
```

```

//Task<ushort[]> result = master.ReadHoldingRegistersAsync(5, 0, 10);
//异步第一种处理，所在方法不需要变更异步方法
//registers = result.GetAwaiter().GetResult();
//异步第二种处理，所在方法需要变更异步方法
//ushort[] result = await master.ReadHoldingRegistersAsync(5, 0, 10);
//查看读取
Console.WriteLine($"保持型寄存器值: {string.Join(",", registers)}");

//读取从站5的输入寄存器0~4
ushort[] inputRegisters = master.ReadInputRegisters(5, 0, 4);
//inputRegisters = await master.ReadHoldingRegistersAsync(5, 0, 10);
Console.WriteLine($"输入寄存器值: {string.Join(",", inputRegisters)}");

//读取从站5的线圈状态0~7
bool[] coils = master.ReadCoils(5, 0, 8);
//coils = await master.ReadCoilsAsync(5, 0, 8);
Console.WriteLine($"线圈状态值: {string.Join(",", coils.Select(c=>c.ToString()))}");

//读取从站5的输入线圈0~7
bool[] input = master.ReadInputs(5, 0, 8);
//input = await master.ReadInputsAsync(5, 0, 8);
Console.WriteLine($"输入线圈值: {string.Join(",", input.Select(c =>c.ToString()))}");

// 读取从站5的保持型寄存器8, 9地址下的浮点数字
ushort[] fs = master.ReadHoldingRegisters(5, 8, 2);
byte[] us_bytes_1 = BitConverter.GetBytes(fs[0]);
byte[] us_bytes_2 = BitConverter.GetBytes(fs[1]);

byte[] f_bytes = new byte[4] {
    us_bytes_1[1],
    us_bytes_1[0],
    us_bytes_2[1],
    us_bytes_2[0]
};
Array.Reverse(f_bytes);
float f2 = BitConverter.ToSingle(f_bytes, 0);
Console.WriteLine($"寄存器浮点数字: {f2}");

//向从站5的寄存器5号地址里写入1234    写单个
// 05
master.WriteSingleRegister(
    slaveAddress: 5, //从站地址 (1-247)

```

```

    registerAddress: 5, //寄存器地址
    value: 1234        //写入值
);

//2字节空间
// 1: byte  0~255    sbyte   -256 ~ 255
sbyte v1;
byte v2;
// 2: ushort 0~65535    short  -32768~32767
ushort v3;
short v4;
UInt16 v5;
Int16 v6;

//向从站5的寄存器0号地址, 1号地址, 2号地址中依次写入三个数据
ushort[] values = { 100, 200, 300 };
master.WriteMultipleRegisters(1, 0, values);

//向从站5的寄存器8号地址, 9号地址写入一个浮点型数字4.5
float v7 = 4.5f; // byte[4]
ushort[] datas = new ushort[2];

//获取浮点型数字对应的字节信息
byte[] v7_bytes = BitConverter.GetBytes(v7);
//// 大小端字节序
//// 0x01 0x2C 大端
//datas[0] = 0x01;
//datas[1] = 0x2C;
//// 0x01 0x2C 小端
//datas[0] = 0x2C;
//datas[1] = 0x01;
// 符合Modbus协议的基本要求 字节序调整, 但是最终以设备方要求为准
Array.Reverse(v7_bytes);

ushort s1 = (ushort)(v7_bytes[0] * 256 + v7_bytes[1]);
s1 = BitConverter.ToUInt16(new byte[] { v7_bytes[1], v7_bytes[0] });

ushort s2 = (ushort)(v7_bytes[2] * 256 + v7_bytes[3]);
s2 = BitConverter.ToUInt16(new byte[] { v7_bytes[3], v7_bytes[2] });

datas[0] = s1;
datas[1] = s2;

master.WriteMultipleRegisters(5, 8, datas);

//向从站5的0号线圈写入True状态

```

```

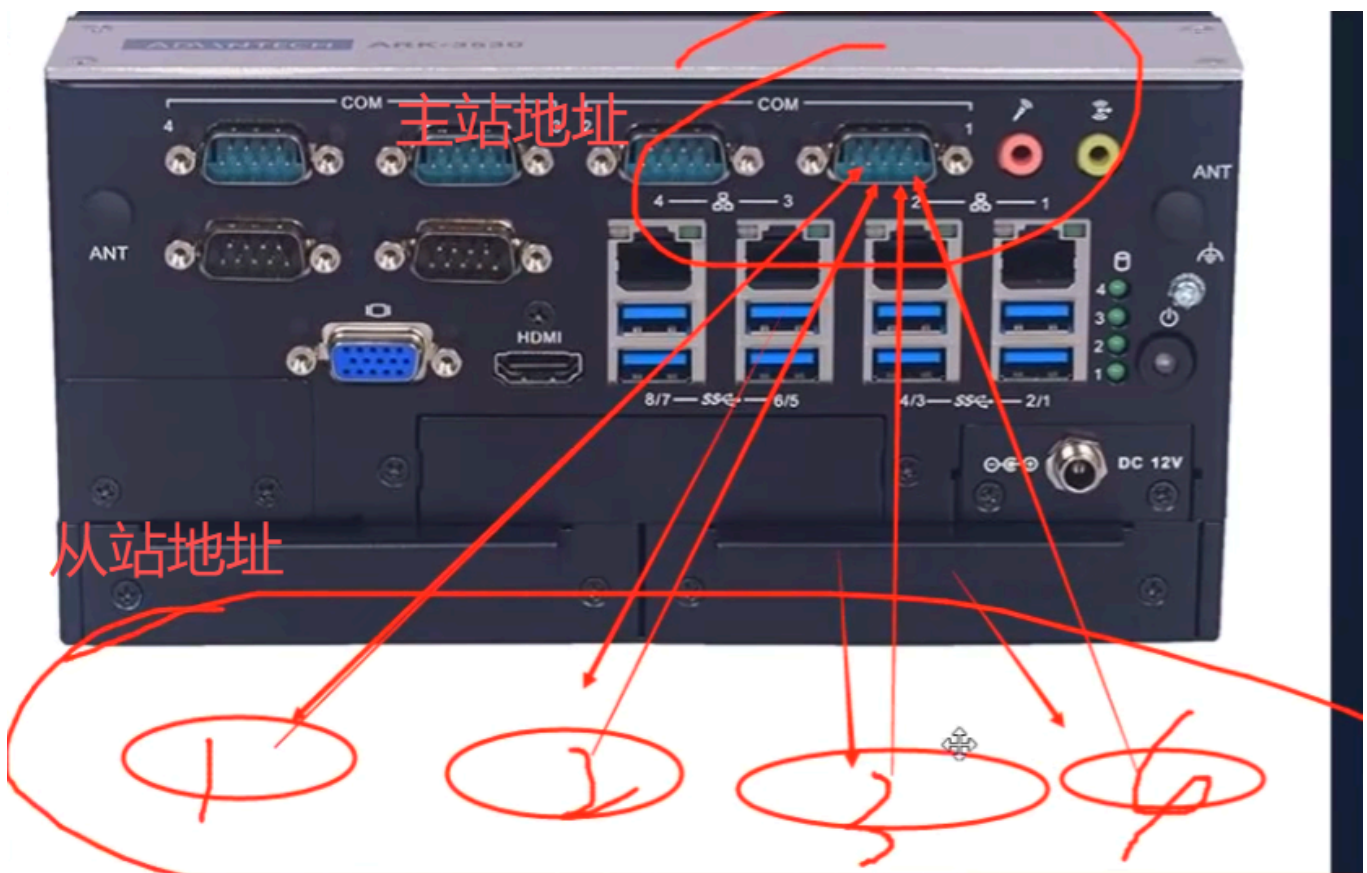
master.WriteSingleCoil(5, 0, true);
//向从站5的5号线圈写入5个状态
master.WriteMultipleCoils(
    5,
    5,
    new bool[] { true, false, true, false, true }
);

```

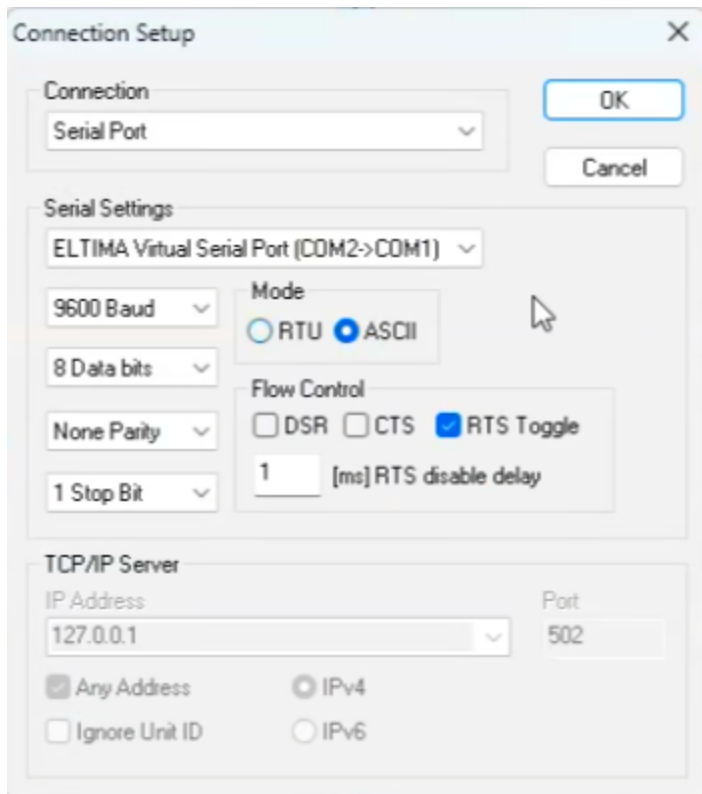
```

//异步写单个线圈状态
//master.WriteSingleCoilAsync();
//异步写多个线圈状态
//master.WriteMultipleCoilsAsync();
//异步写单个寄存器
//master.WriteSingleRegisterAsync();
//异步写多个寄存器
//master.WriteMultipleRegistersAsync();

```



4.ModbusASCII



```
private static async Task NModbus4_ModbusASCII_Test()
{
    SerialPort serialPort = new SerialPort(
        "COM1",
        9600,
        Parity.None,
        8,
        StopBits.One
    );
    serialPort.Open();

    //创建ModbusRTU通信协议对象
    IModbusSerialMaster master = ModbusSerialMaster.CreateAscii(serialPort);

    master.Transport.ReadTimeout = 2000; //读超时
    master.Transport.Retries = 3; //重试次数

    //读取从站5的保持型寄存器0~9
    ushort[] registers = master.ReadHoldingRegisters(
        slaveAddress: 5, //从站地址 (1-247)
        startAddress: 0, //起始寄存器地址
        numberOfPoints: 10 //读取寄存器数量
    );

    //异步读取
    //Task<ushort[]> result = master.ReadHoldingRegistersAsync(5, 0, 10);
```

```

//异步第一种处理，所在方法不需要变更异步方法
//registers = result.GetAwaiter().GetResult();
//异步第二种处理，所在方法需要变更异步方法
//ushort[] result = await master.ReadHoldingRegistersAsync(5, 0, 10);
//查看读取
Console.WriteLine($"保持型寄存器值: {string.Join(",", registers)}");

//读取从站5的输入寄存器0~4
ushort[] inputRegisters = master.ReadInputRegisters(5, 0, 4);
//inputRegisters = await master.ReadHoldingRegistersAsync(5, 0, 10);
Console.WriteLine($"输入寄存器值: {string.Join(",", inputRegisters)}");

//读取从站5的线圈状态0~7
bool[] coils = master.ReadCoils(5, 0, 8);
//coils = await master.ReadCoilsAsync(5, 0, 8);
Console.WriteLine($"线圈状态值: {string.Join(",", coils.Select(c =>
c.ToString()))}");

//读取从站5的输入线圈0~7
bool[] input = master.ReadInputs(5, 0, 8);
//input = await master.ReadInputsAsync(5, 0, 8);
Console.WriteLine($"输入线圈值: {string.Join(",", input.Select(c =>
c.ToString()))}");

//向从站5的寄存器5号地址里写入1234    写单个
// 05
master.WriteSingleRegister(
    slaveAddress: 5, //从站地址 (1-247)
    registerAddress: 5, //寄存器地址
    value: 1234      //写入值
);

//向从站5的寄存器0号地址，1号地址，2号地址中依次写入三个数据
//2字节空间
// 1: byte  0~255    sbyte   -256 ~ 255
sbyte v1;
byte v2;
// 2: ushort 0~65535    short  -32768~32767
ushort v3;
short v4;
UInt16 v5;
Int16 v6;
ushort[] values = { 100, 200, 300 };
master.WriteMultipleRegisters(1, 0, values);

```

```

// 4:int32 uint32 float 单精度浮点
// 寄存器一个地址2byte
float v7 = 4.5f; // byte[4]
ushort[] datas = new ushort[2];
//获取浮点型数字对应的字节信息
byte[] v7_bytes = BitConverter.GetBytes(v7);
//// 大小端字节序
//// 0x01 0x2C 大端
//datas[0] = 0x01;
//datas[1] = 0x2C;
//// 0x01 0x2C 小端
//datas[0] = 0x2C;
//datas[1] = 0x01;
// 符合Modbus协议的基本要求 字节序调整，但是最终以设备方要求为准
Array.Reverse(v7_bytes);

ushort s1 = (ushort)(v7_bytes[0] * 256 + v7_bytes[1]);
s1 = BitConverter.ToUInt16(new byte[] { v7_bytes[1], v7_bytes[0] });

ushort s2 = (ushort)(v7_bytes[2] * 256 + v7_bytes[3]);
s2 = BitConverter.ToUInt16(new byte[] { v7_bytes[3], v7_bytes[2] });

datas[0] = s1;
datas[1] = s2;

master.WriteMultipleRegisters(5, 8, datas);

ushort[] fs = master.ReadHoldingRegisters(5, 8, 2);
byte[] us_bytes_1 = BitConverter.GetBytes(fs[0]);
byte[] us_bytes_2 = BitConverter.GetBytes(fs[1]);

byte[] f_bytes = new byte[4] {
    us_bytes_1[1],
    us_bytes_1[0],
    us_bytes_2[1],
    us_bytes_2[0]
};
Array.Reverse(f_bytes);
float f2 = BitConverter.ToSingle(f_bytes, 0);

//向从站5的0号线圈写入True状态
master.WriteSingleCoil(5, 0, true);
//向从站5的5号线圈写入5个状态
master.WriteMultipleCoils(
    5,

```

```
5,
new bool[] { true, false, true, false, true }
);

//异步写单个线圈状态
//master.WriteSingleCoilAsync();
//异步写多个线圈状态
//master.WriteMultipleCoilsAsync();
//异步写单个寄存器
//master.WriteSingleRegisterAsync();
//异步写多个寄存器
//master.WriteMultipleRegistersAsync();
}
```

5.ModbusRTU vs ASCII 对比

特性	Modbus RTU	Modbus ASCII
传输效率	高 (二进制编码)	低 (ASCII字符)
可读性	差 (需解析十六进制)	高 (直接显示字符)
校验方式	CRC-16	LRC
适用场景	高速、实时性要求高的工业环境	调试阶段或低速率通信

网口通信与协议（以太网）

一、网口通信学习环境

环境准备，下载网络调试助手



二、网口通信对象

1.Socket

Socket是网络通信的核心编程接口，支持TCP、UDP等协议，允许应用程序通过IP地址和端口与其他设备通信。

关键概念：

- 地址族 (AddressFamily)：InterNetwork (IPv4)、InterNetworkV6 (IPv6)。
- 套接字类型(SocketType)：如 Stream(TCP)、Dgram(UDP)。
- 协议类型(ProtocolType)：如Tcp、Udp。
- IPEndPoint：表示网络终结点(IP地址+端口)。

Socket工作流程

步骤	说明
创建 Socket	初始化 Socket 对象，指定地址族、套接字类型和协议类型
绑定 (Bind)	服务器端绑定到本地 IP 和端口 (<code>Bind()</code>)
监听 (Listen)	服务器端开始监听连接请求 (仅 TCP)
连接 (Connect)	客户端连接到服务器 (仅 TCP)
发送 (Send)	发送数据到远程设备
接收 (Receive)	接收来自远程设备的数据
关闭 (Close)	释放资源并关闭 Socket

TCP服务器

```
static void SocketServerTest()
{
    //1.对象创建 Socket
    Socket server = new Socket(
        AddressFamily.InterNetwork, //可选，默认IPv4，可以省略
        SocketType.Stream,
        ProtocolType.Tcp
    );

    //2.绑定本地IP和端口
    IPEndPoint localEndPoint = new IPEndPoint(IPAddress.Any, 8888);
    server.Bind(localEndPoint);

    //3.开启监听,最大等待连接数设为 10
    //** 并不是只能连10个客户端，可以设置更大的值 **
    server.Listen(10);
    Console.WriteLine("服务器已启动，等待客户端连接...");

    //4.接收客户端连接
    while (true)
    {
        Socket client = server.Accept();
        Console.WriteLine($"客户端已连接:{client.RemoteEndPoint}");

        //5.接收数据
        byte[] buffer = new byte[1024];
        Task.Factory.StartNew(() =>
        {
            while (true)
            {
```

```

        int bytesRead = client.Receive(buffer);
        string message = Encoding.UTF8.GetString(buffer, 0,
bytesRead);

        Console.WriteLine($"客户端发送的数据:{message}");

        //6.发送响应
        string response = "Hello WooToot! ";
        client.Send(Encoding.UTF8.GetBytes(response));
    }
});
}

//7.关闭连接
//client.Close();
//server.Close();
}

```

TCP客户端

```

static void SocketClienttest()
{
    //1.创建 Socket 对象
    Socket clientSocket = new Socket(
        AddressFamily.InterNetwork, //可选，默认IPV4，可以省略
        SocketType.Stream,
        ProtocolType.Tcp
    );

    //2.连接到服务器
    IPEndPoint serverEndPoint = new IPEndPoint(IPAddress.Parse("127.0.0.1"),
8888);
    //TCP连接的三次握手（对应的：断开的四次挥手）
    clientSocket.Connect(serverEndPoint);
    Console.WriteLine($"客户端已连接到服务器:{serverEndPoint}");

    //3.发送数据
    string message = "Hello Server!";
    byte[] msg_bytes = Encoding.UTF8.GetBytes(message);
    clientSocket.Send(msg_bytes);

    //4.接收响应
    byte[] buffer = new byte[1024];
    int bytesRead = clientSocket.Receive(buffer);
    string response = Encoding.UTF8.GetString(buffer, 0, bytesRead);
    Console.WriteLine($"服务器响应数据:{response}");
}

```

```

buffer = new byte[1024];
bytesRead = clientSocket.Receive(buffer);
response = Encoding.UTF8.GetString(buffer, 0, bytesRead);
Console.WriteLine($"服务器响应数据:{response}");

// (1) 长数据的接收 做长度的标记
// (2) 多余的数据 可以再接收

//5.关闭连接
//clientSocket.Close();
}

```

UDP通信

```

static void SocketUdpTest()
{
    // UDP 服务端
    Socket udp = new Socket(
        AddressFamily.InterNetwork,
        SocketType.Dgram,
        ProtocolType.Udp
    );

    udp.Bind(new IPEndPoint(IPAddress.Any, 9999));

    //1.指定地址
   EndPoint endPoint = new IPEndPoint(IPAddress.Parse("127.0.0.1"), 9999);
    udp.SendTo(Encoding.UTF8.GetBytes("Hello, UDP!"), endPoint);
    //接收
    byte[] buffer = new byte[1024];
    endPoint = new IPEndPoint(IPAddress.Any, 0);
    //参数接收数据的来源
    int bytesRead = udp.ReceiveFrom(buffer, ref endPoint);
    string message = Encoding.UTF8.GetString(buffer, 0, bytesRead);
    Console.WriteLine($"接收到来自{endPoint}的消息:{message}");

    //2.广播模式 UDP 客户端
    udp.SetSocketOption(SocketOptionLevel.Socket, SocketOptionName.Broadcast,
        true);
    endPoint = new IPEndPoint(IPAddress.Parse("255.255.255.255"), 12345);
    udp.SendTo(Encoding.UTF8.GetBytes("Hello, UDP!-Broadcast"), endPoint);
}

```

2.TcpClient

TcpClient是System.Net.Sockets命名空间下的一个类，封装了底层Socket操作，简化了TCP协议的通信流程。它支持同步和异步操作，适用于客户端和服务端开发。

核心方法及属性

方法/属性	说明
Connect()	连接到远程主机 (IP 地址 + 端口)
GetStream()	获取用于读写的 NetworkStream 对象
Close() / Dispose()	关闭连接并释放资源
Connected	获取当前连接状态 (是否已连接)
Client	获取底层 Socket 对象 (高级操作)

Tcp客户端

```
static void TcpClientTest()
{
    //1.创建 TcpClient 并连接到服务器
    TcpClient client = new TcpClient();

    client.Connect("127.0.0.1", 8888);
    Console.WriteLine($"已连接到服务器");

    //2.获取网络流
    NetworkStream stream = client.GetStream();

    //3.发送数据
    string message = "Hello Server!";
    byte[] sendData = Encoding.UTF8.GetBytes(message);
    stream.Write(sendData, 0, sendData.Length);
    Console.WriteLine($"已发送数据:{message}");

    //4.接收响应
    byte[] buffer = new byte[1024];
    int bytesRead = stream.Read(buffer, 0, buffer.Length);
    string response = Encoding.UTF8.GetString(buffer, 0, bytesRead);
    Console.WriteLine($"接收到响应数据:{response}");
}
```

Tcp服务端

```
static void TcpListenerTest()
{
    TcpListener server = new TcpListener(IPAddress.Any, 8888);
    server.Start();
}
```

```

Console.WriteLine("服务器已启动，等待客户端连接...");

try
{
    //1.接受客户端连接
    using (TcpClient client = server.AcceptTcpClient())
    {
        Console.WriteLine($"客户端已连接:
{((IPEndPoint)client.Client.RemoteEndPoint).Address}");

        //2.获取网络流
        NetworkStream stream = client.GetStream();

        //3.接收数据
        byte[] buffer = new byte[1024];
        int bytesRead = stream.Read(buffer, 0, buffer.Length);
        string message = Encoding.UTF8.GetString(buffer, 0, bytesRead);
        Console.WriteLine($"接收到响应数据:{message}");

        //4.发送响应
        string response = "Hello Client!";
        byte[] sendData = Encoding.UTF8.GetBytes(response);
        stream.Write(sendData, 0, sendData.Length);
        Console.WriteLine($"已发送数据:{response}");
    }
}
catch (Exception ex)
{
    Console.WriteLine($"错误: {ex.Message}");
}
finally
{
    server.Stop();
    Console.WriteLine("服务器已停止");
}
}

```

3.常见问题与调试

- 连接失败
 - 检查IP和端口:确保服务器监听正确端口，客户端使用正确IP
 - 防火墙设置:允许应用程序通过防火墙。
 - 网络可达性:使用ping或telnet 测试连通性。
- 数据接收不完整

- 循环接收:持续调用Receive直到数据完整
- 缓冲区管理:动态调整缓冲区大小或使用内存流。
- 性能优化
 - 重用Socket:设置 ReuseAddress 选项。
 - 异步模式:使用 SocketAsyncEventArgs 高性能API。
 - 批量发送:合并小数据包减少系统调用。

三、通信协议

1.ModbusTCP

```
static void ModbusTCP_Test()
{
    TcpClient tcpClient = new TcpClient();
    tcpClient.Connect("127.0.0.1", 502);
    //创建ModbusRTU通信协议对象
    ModbusMaster master = ModbusIpMaster.CreateIp(tcpClient);

    master.Transport.ReadTimeout = 2000; //读超时
    master.Transport.Retries = 3; //重试次数

    //关于主站读取的相关方法
    //-----
    //读取从站1的保持型寄存器0~9
    ushort[] registers = master.ReadHoldingRegisters(
        slaveAddress: 1, //从站地址 (1-247)
        startAddress: 0, //起始寄存器地址
        numberOfPoints: 10 //读取寄存器数量
    );

    //异步读取
    //Task<ushort[]> result = master.ReadHoldingRegistersAsync(1, 0, 10);
    //异步第一种处理, 所在方法不需要变更异步方法
    //registers = result.GetAwaiter().GetResult();
    //异步第二种处理, 所在方法需要变更异步方法 (需要方法改成async Task返回)
    //ushort[] result = await master.ReadHoldingRegistersAsync(1, 0, 10);
    //查看读取
    Console.WriteLine($"保持型寄存器值: {string.Join(", ", registers)}");

    //读取从站1的输入寄存器0~4
    ushort[] inputRegisters = master.ReadInputRegisters(1, 0, 4);
    //inputRegisters = await master.ReadHoldingRegistersAsync(5, 0, 10);
    Console.WriteLine($"输入寄存器值: {string.Join(", ", inputRegisters)}");
}
```

```

//读取从站1的线圈状态0~7
bool[] coils = master.ReadCoils(1, 0, 8);
//coils = await master.ReadCoilsAsync(5, 0, 8);
Console.WriteLine($"线圈状态值: {string.Join(",", coils.Select(c =>
c.ToString()))}");

//读取从站1的输入线圈0~7
bool[] input = master.ReadInputs(1, 0, 8);
//input = await master.ReadInputsAsync(5, 0, 8);
Console.WriteLine($"输入线圈值: {string.Join(",", input.Select(c =>
c.ToString()))}");


//关于主站写入的相关方法
//-----
//向从站的寄存器5号地址里写入1234    写单个
master.WriteSingleRegister(
    slaveAddress: 1, //从站地址 (1-247)
    registerAddress: 5, //寄存器地址
    value: 1234      //写入值
);

//向从站1的寄存器0号地址, 1号地址, 2号地址中依次写入三个数据
ushort[] values = { 100, 200, 300 };
master.WriteMultipleRegisters(1, 0, values);

//向从站1的0号线圈写入True状态
master.WriteSingleCoil(1, 0, true);
//向从站1的5号线圈写入5个状态
master.WriteMultipleCoils(
    1,
    5,
    new bool[] { true, false, true, false, true }
);

//异步写单个线圈状态
//master.WriteSingleCoilAsync();
//异步写多个线圈状态
//master.WriteMultipleCoilsAsync();
//异步写单个寄存器
//master.WriteSingleRegisterAsync();
//异步写多个寄存器
//master.WriteMultipleRegistersAsync();
}

```

2.西门子S7

西门子PLC的**S7协议**(又称S7COMM)是专为S7系列PLC设计的工业通信协议，支持高效稳定的数据交换。S7协议基于**OSI模型**，主要工作在**传输层(TCP)**和应用层：

- **传输层**：使用TCP(端口102)，保障可靠数据传输。
- **应用层**：定义功能码(Function Code)和数据包格式，实现具体操作(如读写数据块)

基本通信环境



仿真环境安装

[百度网盘](#) 请输入提取码